

Third Normal Form (3NF), BCNF and Final Database Design

1. Objective

The objective of Week 6 is to:

1. Apply **Third Normal Form (3NF)** to eliminate transitive dependencies.
2. Apply **Boyce-Codd Normal Form (BCNF)** to make sure every determinant is a candidate key.
3. Produce a **finalized database design** ready for implementation.

2. Definitions

Normal form	Rule
3NF	The table is in 2NF and no non-key attribute depends on another non-key attribute (no transitive dependency)
BCNF	For every functional dependency $X \rightarrow Y$ in the table, X must be a candidate key (a stricter form of 3NF)

3. Third Normal Form (3NF)

3.1 Schema Before 3NF (2NF)

ORDERS (Order_ID, Order_Date, Order_Status, Customer_ID, Cust_Name, Cust_Email, Cust_Mobile, Street, City, State, Pincode, Payment_Method, Payment_Status, Shipment_Status)

PRODUCT (Product_ID, Product_Name, Price, Category_ID, Category_Name, Supplier_ID, Supplier_Name)

ORDER_DETAILS (Order_ID, Product_ID, Quantity, Unit_Price)

3.2 Transitive Dependencies Found

Table	Transitive dependency	Problem
ORDERS	Order_ID $\rightarrow$ Customer_ID $\rightarrow$ Cust_Name, Cust_Email, Cust_Mobile, Street, City, State, Pincode	Customer details repeat in every order of that customer
ORDERS	Order_ID $\rightarrow$ Payment_Method, Payment_Status and Order_ID $\rightarrow$ Shipment_Status	These facts describe the payment and the shipment, which are separate entities with their own attributes (Payment_Date,

Table	Transitive dependency	Problem
		Delivery_Address, Shipment_Date)
PRODUCT	Product_ID → Category_ID → Category_Name	Category name repeats for every product of the category
PRODUCT	Product_ID → Supplier_ID → Supplier_Name	Supplier name repeats for every product of the supplier

3.3 Steps Taken to Reach 3NF

1. Move customer details to a new table **CUSTOMER** with Customer_ID as primary key. Keep Customer_ID in ORDERS as a foreign key.
2. Move payment facts to **PAYMENT** and shipment facts to **SHIPMENT**, each with its own primary key and Order_ID as a foreign key.
3. Move category details to **CATEGORY**. Keep Category_ID in PRODUCT as a foreign key.
4. Move supplier details to **SUPPLIER**.
5. Remove Supplier_ID from PRODUCT. Because a product can have several suppliers and a supplier can supply several products (many-to-many), create the linking table **PRODUCT_SUPPLIER**.
6. Include the entity **REVIEW**, which records customer ratings of products.

3.4 Sample Data After 3NF

CUSTOMER

Customer_ID	Name	City	Pincode
C01	Arun	Trichy	620017
C02	Divya	Chennai	600001

CATEGORY

Category_ID	Category_Name
CAT1	Electronics

PRODUCT

Product_ID	Product_Name	Price	Category_ID
P01	Laptop	55000	CAT1
P02	Mouse	500	CAT1

ORDERS

Order_ID	Customer_ID	Order_Date	Order_Status
1001	C01	2026-09-01	Delivered
1002	C01	2026-09-05	Shipped
1003	C02	2026-09-07	Placed

Arun's details are stored once, and the name "Electronics" is stored once. Changing either needs only a single edit.

4. Boyce-Codd Normal Form (BCNF)

4.1 BCNF Check on Every Table

For each table, every determinant must be a candidate key.

Table	Candidate keys	Functional dependencies	BCNF?
CUSTOMER	Customer_ID, Email, Mobile	Customer_ID → all attributes; Email → all; Mobile → all	Yes, all determinants are candidate keys
CATEGORY	Category_ID	Category_ID → Category_Name, Description	Yes
SUPPLIER	Supplier_ID	Supplier_ID → Supplier_Name, Contact_Info	Yes
PRODUCT	Product_ID	Product_ID → Product_Name, Price, Stock_Quantity, Category_ID	Yes
PRODUCT_SUPPLIER	(Product_ID, Supplier_ID)	(Product_ID, Supplier_ID) → Supply_Price	Yes
ORDERS	Order_ID	Order_ID → Customer_ID, Order_Date, Total_Amount, Order_Status	Yes
ORDER_DETAILS	Order_Detail_ID,	Both keys → Quantity,	Yes

Table	Candidate keys	Functional dependencies	BCNF?
	(Order_ID, Product_ID)	Unit_Price	
PAYMENT	Payment_ID, Order_ID	Payment_ID → Order_ID, Payment_Method, Payment_Date, Payment_Status; Order_ID → Payment_ID	Yes (one payment record per order)
SHIPMENT	Shipment_ID, Order_ID	Shipment_ID → Order_ID, Delivery address fields, Shipment_Date, Delivery_Status; Order_ID → Shipment_ID	Yes (one shipment record per order)
REVIEW	Review_ID, (Customer_ID, Product_ID)	Both keys → Rating, Comments	Yes (one review per customer per product)

No table has a determinant that is not a candidate key, so **all tables are in BCNF**.

4.2 Why Customer Needs Care

In CUSTOMER, Email → Name holds, but Email is a candidate key (each customer has a unique email), so there is no BCNF violation. If Email were not unique, the table would have to be split.

5. Design Decisions and Accepted Trade-offs

No.	Decision	Reason
1	Unit_Price kept in ORDER_DETAILS although PRODUCT has Price	It records the price at the time of purchase, so later price changes do not alter old orders. It depends on the order line, so it is not redundant
2	Total_Amount kept in ORDERS	It is derived from $\Sigma(\text{Quantity} \times \text{Unit_Price})$, so it is deliberate redundancy kept for quick access. It must always match its order lines

No.	Decision	Reason
3	Delivery address kept in SHIPMENT although CUSTOMER has an address	A customer may ship to a different address, so these are different facts
4	Pincode → City, State is not split into its own table	Strictly, Pincode determines City and State. In real data one pincode can cover several localities and city names, so the dependency is not reliable, and a separate table would add extra lookups for little gain. This is the one documented exception to strict 3NF
5	Surrogate key Order_Detail_ID in ORDER_DETAILS	Gives a simple single-column key. (Order_ID, Product_ID) remains a candidate key so the same product is not repeated in one order

6. Final Normalized Database Design

6.1 Relational Schema (Primary Key in bold, Foreign Key marked FK)

No.	Table	Attributes
1	CUSTOMER	Customer_ID , Name, Email, Mobile, Street, City, State, Pincode, Registration_Date
2	CATEGORY	Category_ID , Category_Name, Description
3	SUPPLIER	Supplier_ID , Supplier_Name, Contact_Info
4	PRODUCT	Product_ID , Product_Name, Price, Stock_Quantity, Category_ID (FK)
5	PRODUCT_SUPPLIER	Product_ID (FK), Supplier_ID (FK), Supply_Price
6	ORDERS	Order_ID , Customer_ID (FK), Order_Date, Total_Amount,

No.	Table	Attributes
		Order_Status
7	ORDER_DETAILS	Order_Detail_ID , Order_ID (FK), Product_ID (FK), Quantity, Unit_Price
8	PAYMENT	Payment_ID , Order_ID (FK), Payment_Method, Payment_Date, Payment_Status
9	SHIPMENT	Shipment_ID , Order_ID (FK), Delivery_Street, Delivery_City, Delivery_State, Delivery_Pincode, Shipment_Date, Delivery_Status
10	REVIEW	Review_ID , Customer_ID (FK), Product_ID (FK), Rating, Comments

6.2 Candidate (Alternate) Keys

Table	Alternate key
CUSTOMER	Email; Mobile
ORDER_DETAILS	(Order_ID, Product_ID)
PAYMENT	Order_ID
SHIPMENT	Order_ID
REVIEW	(Customer_ID, Product_ID)

6.3 Relationships

Parent table	Child table	Foreign key	Type
CUSTOMER	ORDERS	Customer_ID	One-to-Many
CUSTOMER	REVIEW	Customer_ID	One-to-Many
CATEGORY	PRODUCT	Category_ID	One-to-Many
PRODUCT	ORDER_DETAILS	Product_ID	One-to-Many
PRODUCT	REVIEW	Product_ID	One-to-Many
ORDERS	ORDER_DETAILS	Order_ID	One-to-Many

Parent table	Child table	Foreign key	Type
ORDERS	PAYMENT	Order_ID	One-to-One
ORDERS	SHIPMENT	Order_ID	One-to-One
PRODUCT and SUPPLIER	PRODUCT_SUPPLIER	Product_ID, Supplier_ID	Many-to-Many (resolved by PRODUCT_SUPPLIER)
ORDERS and PRODUCT	ORDER_DETAILS	Order_ID, Product_ID	Many-to-Many (resolved by ORDER_DETAILS)

7. Anomalies Resolved

Anomaly	Status in final design
Cannot add a product without an order	Fixed: PRODUCT is independent
Cannot register a customer who has not ordered	Fixed: CUSTOMER is independent
Cannot add a category with no products	Fixed: CATEGORY is independent
Deleting an order loses customer details	Fixed: customer stored in CUSTOMER
Deleting an order loses product price	Fixed: product stored in PRODUCT
Address changed in only some rows	Fixed: address stored once
Category renamed in many rows	Fixed: name stored once in CATEGORY
Product price change alters old orders	Fixed: history kept in ORDER_DETAILS.Unit_Price

8. Conclusion

The 2NF schema was brought to 3NF by removing transitive dependencies through Customer, Payment, Shipment, Category and Supplier, and the many-to-many link between Product and Supplier was resolved with PRODUCT_SUPPLIER. All ten tables were then checked against BCNF and every determinant was found to be a candidate key. Each fact is now stored once, and the insertion, deletion and update anomalies are removed. The final design of ten tables, with their primary keys, foreign keys and relationships, is ready for implementation.